

Competitive Programming

Section 47 Editorial: Meet-In-The-Middle

Competitive Programming Course

2026-07-18

Table of contents

1 Editorial overview	3
1.1 Problem-specific proof map	3
1.2 Detailed solution guide	3
2 A. Count Subset Sums $\leq X$	4
2.1 Restatement	4
2.2 Algorithm and proof	4
2.3 Complexity	4
2.4 Full C++ solution	4
2.5 Full Python solution	4
3 B. Best Subset Sum $\leq X$	5
3.1 Restatement	5
3.2 Algorithm and proof	5
3.3 Complexity	5
3.4 Full C++ solution	5
3.5 Full Python solution	5
4 C. Closest Subset Sum	6
4.1 Restatement	6
4.2 Algorithm and proof	6
4.3 Complexity	6
4.4 Full C++ solution	6
4.5 Full Python solution	6
5 D. Exact Sum, Maximum Items	7
5.1 Restatement and constraint implications	7
5.2 State and worked trace	7
5.3 Algorithm	7
5.4 Correctness proof	8
5.5 Complexity, implementation notes, and tests	8
5.6 Full C++ solution	8
5.7 Full Python solution	9
6 Testing discussion	11

1 Editorial overview

1.1 Problem-specific proof map

A. Count Subset Sums $\leq X$. Every subset is uniquely a left subset plus a right subset. Sorting right sums lets each left sum count compatible partners with `upper_bound`.

B. Best Subset Sum $\leq X$. The same unique split applies; binary-search the greatest right sum no larger than `X-left` and maximize over left sums.

C. Closest Subset Sum. The closest right partner to `target-left` is one of the two sorted neighbors around a lower bound, so checking both covers every optimal full subset.

D. Exact Sum, Maximum Items. For each right-half sum, store only the largest number of selected items producing it. A left state of sum `s` and count `c` pairs with the stored state for `target-s`, maximizing `c+count`.

1.2 Detailed solution guide

A. Count Subsets with Sum at Most `S`. Split the array into left and right halves. Enumerate every subset sum of each half. For a fixed left sum `x`, the valid right sums are `y ≤ S - x`; after sorting the right sums, their count is `upper_bound(right, S-x)`. Every original subset has exactly one left part and one right part, so the sum over all left sums counts each valid subset once.

B. Maximum Subset Sum at Most `S`. Use the same split and sorted right sums. For each left sum `x ≤ S`, find the largest right sum `y ≤ S-x` and maximize `x+y`. This is the optimization version of A: instead of counting all compatible partners, keep the best compatible partner.

C. Closest Subset Sum. Enumerate both halves. For each left sum `x`, the best right partner is near `target - x` in sorted order. Check `lower_bound` and its predecessor, because one is the first sum not below the target gap and the other is the largest sum below it. One of those two minimizes the absolute difference for this `x`; scanning all `x` gives the global optimum.

Common pitfalls. Include the empty subset in both halves. Use 64-bit integers for sums. The enumeration cost is $2^{(n/2)}$, which is the whole point of the technique.

Meet-in-the-middle splits one exponential choice into two half-sized enumerations.

- A counts pairs of half-subset sums whose total is at most `X`.
- B maximizes such a total instead of counting it.
- C checks the two right-half sums nearest to `T-left_sum`.

Every full subset is uniquely determined by its left half and right half, so enumerating both lists is complete. Sorting one side avoids trying all pairs.

2 A. Count Subset Sums $\leq X$

2.1 Restatement

Count subsets whose sum is at most X .

2.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

2.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<long long>sums(vector<long long>a){vector<long long>r{0};for(lon
```

2.5 Full Python solution

```
import sys,bisect
def sums(a):
    res=[0]
    for x in a: res += [y+x for y in res]
    return res
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,x=data[0],data[1]; a=data[2:]; left=sums(a[:n//2]); right=sorted(sums(a[n//2:])); ans=
    for s in left: ans+=bisect.bisect_right(right,x-s)
    print(ans)
if __name__=="__main__": main()
```

3 B. Best Subset Sum $\leq X$

3.1 Restatement

Print the maximum subset sum not exceeding X .

3.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

3.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<long long>sums(vector<long long>a){vector<long long>r{0};for(lon
```

3.5 Full Python solution

```
import sys,bisect
def sums(a):
    res=[0]
    for x in a: res += [y+x for y in res]
    return res
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,x=data[0],data[1]; a=data[2:]; left=sums(a[:n//2]); right=sorted(sums(a[n//2:])); best=0
    for s in left:
        i=bisect.bisect_right(right,x-s)-1
        if i>=0: best=max(best,s+right[i])
    print(best)
if __name__=="__main__": main()
```

4 C. Closest Subset Sum

4.1 Restatement

Print the minimum absolute difference between a subset sum and target T.

4.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

4.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<long long>sums(vector<long long>a){vector<long long>r{0};for(lon
```

4.5 Full Python solution

```
import sys,bisect
def sums(a):
    res=[0]
    for x in a: res += [y+x for y in res]
    return res
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,t=data[0],data[1]; a=data[2:]; L=sums(a[:n//2]); R=sorted(sums(a[n//2:])); best=10**30
    for s in L:
        i=bisect.bisect_left(R,t-s)
        for j in (i-1,i):
            if 0<=j<len(R): best=min(best,abs(s+R[j]-t))
    print(best)
if __name__=="__main__": main()
```

5 D. Exact Sum, Maximum Items

5.1 Restatement and constraint implications

Choose a subset whose sum is exactly `target`. Among all such subsets, print the maximum number of selected items, or `-1` when no exact-sum subset exists. There are at most 36 values, which may be negative. This makes a target-indexed dynamic program impossible when values and target are large, while direct subset enumeration remains exponential in 36.

The split is the same as other meet-in-the-middle tasks, but the data retained for an equal sum changes. We do not merely need to know whether a right-half sum exists: we need the largest item count that achieves that sum.

5.2 State and worked trace

For values `[4,3,2,1,-1]` and target 5, split after the first two values. The left half contributes states `(sum,count)` including `(0,0)`, `(4,1)`, `(3,1)`, and `(7,2)`. The right half `[2,1,-1]` has, among others, sum 1 with maximum count 3, from selecting all three values. Pairing left sum 4 with right sum `5-4=1` gives four selected items: `[4,2,1,-1]`.

If several right subsets have the same sum, retaining only the largest count is safe. Any left state needing that sum benefits from the largest count; a smaller count can never improve the final objective.

5.3 Algorithm

```
enumerate (sum, count) for every subset of the right half
best_count[sum] = maximum count among its right-half states

answer = no solution
for every (left_sum, left_count) from the left half:
    if target - left_sum exists in best_count:
        answer = max(answer, left_count + best_count[target-left_sum])
print answer, or -1 if it was never updated
```

The enumeration must include `(0,0)` for each half, which represents taking no item from that half. Negative values require no pruning by partial sum.

5.4 Correctness proof

Every full subset has a unique left-half subset and right-half subset. Therefore it appears exactly once as a pair of enumerated states whose sums add to its total. For a fixed left state with sum s , an exact target is possible exactly when the right sum is $\text{target} - s$.

For that required right sum, `best_count` stores the maximum item count among all right subsets with that sum. Replacing any compatible right subset by that stored one preserves the exact total and cannot decrease item count. Thus the algorithm considers an item count at least as large as every valid full subset. It never combines sums other than `target`, so every candidate is valid. Consequently its maximum is precisely the largest cardinality of an exact-sum subset, or no candidate exists exactly when no such subset exists. QED.

5.5 Complexity, implementation notes, and tests

Each half has at most 18 elements, so its enumeration contains at most 2^{18} states. Expected dictionary operations make the total time $O(2^{(n/2)})$ and extra memory $O(2^{(n/2)})$. Use signed 64-bit integers for sums and targets in C++; Python integers already handle them.

Test target zero, all negative values, repeated values that create the same sum with different cardinalities, and an impossible target. A frequent bug is to overwrite a map entry with the last count rather than the maximum count.

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using State = pair<long long, int>;

vector<State> enumerate_states(const vector<long long>& values) {
    vector<State> states{{0, 0}};
    states.reserve(1ULL << values.size());
    for (long long value : values) {
        int old_size = (int)states.size();
        for (int i = 0; i < old_size; i++) {
            states.push_back({states[i].first + value, states[i].second + 1});
        }
    }
    return states;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long target;
```

```

if (!(cin >> n >> target)) return 0;
vector<long long> values(n);
for (long long& value : values) cin >> value;

vector<long long> left_values(values.begin(), values.begin() + n / 2);
vector<long long> right_values(values.begin() + n / 2, values.end());
vector<State> left_states = enumerate_states(left_values);
vector<State> right_states = enumerate_states(right_values);

unordered_map<long long, int> best_right_count;
best_right_count.reserve(right_states.size() * 2);
for (const State& state : right_states) {
    best_right_count[state.first] = max(best_right_count[state.first], state.second);
}

int answer = -1;
for (const State& state : left_states) {
    auto it = best_right_count.find(target - state.first);
    if (it != best_right_count.end()) answer = max(answer, state.second + it->second);
}
cout << answer << '\n';
return 0;
}

```

5.7 Full Python solution

```

import sys

def enumerate_states(values: list[int]) -> list[tuple[int, int]]:
    states = [(0, 0)]
    for value in values:
        states.extend((total + value, count + 1) for total, count in states[:])
    return states

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, target = data[0], data[1]
    values = data[2 : 2 + n]
    middle = n // 2
    left_states = enumerate_states(values[:middle])
    right_states = enumerate_states(values[middle:])

    best_right_count: dict[int, int] = {}
    for total, count in right_states:

```

```
previous = best_right_count.get(total, -1)
if count > previous:
    best_right_count[total] = count

answer = -1
for total, count in left_states:
    other_count = best_right_count.get(target - total)
    if other_count is not None:
        answer = max(answer, count + other_count)
print(answer)

if __name__ == "__main__":
    main()
```

6 Testing discussion

The proof is a bijection between full subsets and pairs of half-subsets. For small n , enumerate all masks as the stress oracle. In counting, use `upper_bound(X-left_sum)`; in optimization, verify the candidate immediately below that bound, never merely the first greater value.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.

For D , tiny cases instead enumerate every full-array mask, independently checking exact sum and item count. The scale case uses 36 power-of-two values, which makes a full enumeration impossible while giving a directly known answer.